Cobalt: Simulated US Digital Bank PRD

Working name: Cobalt. Market: United States, USD only. Version 1.1, draft for build.

1. Overview

Cobalt is a simulated US neobank, modeled on Monzo's product design and the Chime-style US partner-bank model: a consumer web app with accounts, instant transfers, spending categorization, savings Pots, round-ups and budgets, backed by a double-entry ledger and an admin console. It is a sandbox product. No real money moves, and there is no real banking license, card network or KYC provider. Everything that would be external in production (funding, card spend, merchants) is simulated through clearly labeled system accounts. Like US neobanks, it is a product layer on top of a partner bank, and nothing in the app claims FDIC insurance.

Goal

Ship a working, deployed, demo-ready banking app that proves the builder can design and build money-moving software correctly, as an application to a fintech company.

What reviewers should conclude

- The money logic is correct: balances never drift, transfers never double-post, concurrent requests are safe.
- The builder understands how a real fintech is structured: ledger first, product layer second, compliance-minded controls.
- The product is polished enough to feel like a real banking app in a 3-minute demo.
- The scope decisions were deliberate and documented.

Non-goals

Real payments, real KYC, cards, crypto, stocks, loans, mobile native apps, multi-tenant bank-as-a-service.

2. Target user and positioning

Primary persona: a US-based young professional who wants a clean everyday account with instant feedback on spending and easy saving. This mirrors the core audience of Monzo in the UK and Chime in the US.

Positioning: the everyday account that makes it obvious where your money goes and makes saving automatic.

Core jobs to be done:

1. Know my balance and what I just spent, instantly.
2. Send money to another person in seconds without mistakes.
3. Set money aside for goals without thinking about it.
4. Stay inside a budget and get warned before I overspend.

3. Success criteria

Product:

- A new user can sign up, get an account, fund it, pay someone, and create a Pot in under 3 minutes.
- Transaction feed updates in real time, with category and merchant, with no manual refresh.

Engineering (hard requirements):

- Sum of all ledger entries per transaction is exactly zero, enforced by the database.
- Sum of all ledger entries across the whole system is zero at all times.
- Every account balance equals the sum of its ledger entries (reconciliation job passes).
- 100 concurrent transfers against one account never overdraw it.
- Repeating the same request with the same idempotency key never creates a second transfer.
- No ledger row is ever updated or deleted.

Delivery:

- Deployed on Vercel with seeded demo data, a README with architecture decisions, and a 3-minute demo video.

4. Scope

In scope (MVP)

Auth, simulated onboarding, current account, double-entry ledger, sandbox top-up, peer transfers, transaction feed with search and categories, Pots, round-ups, budgets and spending insights, notifications, transfer limits and velocity rules, admin console, audit log, statement export.

Explicitly out of scope (documented as conscious decisions)

- Debit cards and card network integration. Card spend is simulated through a Simulate purchase action.
- Real KYC and document verification. Onboarding collects US-style details (legal name, date of birth, US address) and marks the user verified in sandbox mode. No real SSN is ever collected.
- Crypto, stocks, lending, overdrafts.
- Real bank rails (ACH, FedNow, RTP, wires). Top-ups come from a sandbox funding source.
- Native mobile apps. The web app is responsive and installable as a PWA if time allows.

5. Feature list

Priorities: P0 must ship, P1 should ship, P2 only if the core is airtight. Everything P0 must be finished by the end of day 6.

5.1 Authentication and onboarding

- P0: Email and password sign-up and login via Supabase Auth, email verification.
- P0: Simulated onboarding: full name, date of birth, US address (street, city, state, ZIP), choose a unique handle (like a Cash App cashtag) used for payments.
- P0: On completion, create the user's main account with a clearly fake 9-digit routing number, a 12-digit account number and a handle.
- P1: Optional 2FA with TOTP.
- P1: Session management with a list of active sessions and sign-out everywhere.
- P2: Biometric-style PIN lock screen after inactivity.

Acceptance: a user cannot access any banking screen until onboarding is complete; handles are unique and case-insensitive.

5.2 Accounts and balances

- P0: Main checking account in USD (single currency in v1, schema kept currency-aware).
- P0: Balance shown as two values: ledger balance and available balance (ledger minus holds).
- P0: Balance is a cached column updated inside the same database transaction as the ledger entries, with a reconciliation check.
- P1: Account freeze (self-service) and admin freeze. Frozen accounts block debits but still receive credits unless admin-blocked.

Acceptance: balance can never go negative; a reconciliation query comparing cached balance to the sum of entries returns zero discrepancies.

5.3 Ledger (the core)

- P0: Double-entry model. Every money movement is a transaction with two or more entries; debits equal credits.
- P0: Amounts are stored as integer minor units (bigint) with a currency code. No floating point anywhere.
- P0: Entries are append-only. Corrections are made with compensating transactions (reversals), never edits.
- P0: System accounts: External Funding (source of sandbox top-ups), Merchant Settlement (destination of simulated spend), Fees, Suspense.
- P0: A deferred database constraint trigger rejects any transaction whose entries do not sum to zero.
- P0: Every transaction carries a type (transfer, topup, purchase, pot_move, round_up, reversal, fee), a status (pending, posted, failed, reversed) and metadata.

Acceptance: attempting to insert unbalanced entries fails at the database level, not just in application code.

5.4 Funding (sandbox)

- P0: Add money button that creates a top-up from the External Funding system account, with preset amounts and a custom amount up to a sandbox limit.
- P1: Simulated incoming salary on a schedule, to generate realistic demo history.

5.5 Peer transfers

- P0: Send money to another user by handle or account number.
- P0: Confirm screen showing recipient name, amount and reference before sending.
- P0: Executed by a single database function in one transaction: locks both accounts in a consistent order (by account id) to prevent deadlocks, checks balance and account status, writes entries, updates cached balances.
- P0: Idempotency keys. The client sends a key per attempt; the server stores the key with the response, so retries and double-clicks return the original result.
- P0: Immediate success and failure states with clear error messages (insufficient funds, frozen account, limit exceeded, recipient not found).
- P1: Payee list with saved recipients and recent recipients.
- P1: Payment requests: ask another user for money, they can accept or decline.
- P2: Scheduled and recurring transfers, executed by a cron job with the same idempotent path.

Acceptance: 100 parallel transfers from one account with insufficient total funds results in exactly the affordable number succeeding and the balance never below zero.

5.6 Transaction feed

- P0: Reverse-chronological feed with merchant or counterparty, amount, category icon, timestamp and status.
- P0: Transaction detail screen with full metadata, running balance and category.
- P0: Real-time updates via Supabase Realtime.
- P0: Search by merchant, amount range, date range and category, with pagination.
- P1: Manual category override with a remembered rule for that merchant.
- P1: Notes and attachments on a transaction.
- P2: Map or location for simulated merchants.

5.7 Simulated card spend and categorization

- P0: A Simulate purchase panel with a catalog of fictional US-style merchants (grocery, transport, eating out, entertainment, bills) that posts a purchase from the user's account to Merchant Settlement.
- P0: Rule-based auto-categorization by merchant category, instant on posting.
- P1: Random realistic spend generator for seeding demo accounts.

5.8 Pots (savings goals)

- P0: Create a Pot with a name, emoji and optional target amount and date.
- P0: Move money in and out of a Pot. A Pot is its own ledger account owned by the user; moving money is a transfer between two of the user's own accounts.
- P0: Pot balance is excluded from the spendable available balance.
- P1: Progress bar and projected completion date based on contribution history.
- P1: Auto-deposit rules: fixed amount weekly or monthly.
- P2: Shared Pots: invite another user as a member with permission to contribute, with a full contribution history per member.

5.9 Round-ups

- P0: Toggle round-ups on a chosen Pot. After each purchase, the difference to the next whole unit (for example 3.40 becomes 0.60) moves to that Pot as a separate ledger transaction linked to the purchase.
- P0: Round-up never fires if it would overdraw the account.
- P1: Round-up multiplier (2x, 5x).

5.10 Budgets and spending insights

- P0: Monthly spending summary by category with a chart.
- P1: Set a budget per category with a progress bar.
- P1: Alert when a budget reaches 80 percent and 100 percent.
- P1: Predictive alert: projected month-end spend per category based on the trailing average, flagged if it will exceed the budget.
- P2: Month-over-month comparison and top merchants.

5.11 Notifications

- P0: In-app notification center with real-time toasts for money in, money out, failed payments, limit warnings and budget alerts.
- P1: Email notifications for security events (new login, large transfer, account frozen).
- P2: Web push notifications.

5.12 Limits and fraud controls

- P0: Per-transaction limit and daily outgoing limit per account, configurable per user by admin.
- P0: Velocity rule: more than 5 outgoing transfers in 10 minutes is flagged.
- P0: Large-transfer rule: amounts above a threshold, or a first payment to a new payee above a lower threshold, are held in pending state for review.
- P1: Held transfers can be released or declined by an admin; the sender is notified.
- P1: Self-service security settings: lower my own limits, freeze account.
- P2: Simple anomaly flag: spend more than 3x the user's trailing average in one transaction.

5.13 Admin console

- P0: Separate role-protected area. Admin role stored in a table with RLS, never trusted from the client.
- P0: User list with search, account status and balances.
- P0: Review queue for flagged and held transactions with approve and decline actions.
- P0: Freeze and unfreeze accounts, with a mandatory reason.
- P0: System dashboard showing total balances, system ledger sum (must be zero) and the last reconciliation result.
- P1: Issue a reversal on a posted transaction via a compensating entry.
- P1: Adjust user limits.

5.14 Audit log

- P0: Append-only audit log of security and money-relevant actions: login, failed login, transfer attempts, limit changes, freeze, admin actions, reversals. Records actor, action, target, IP, timestamp and before and after values.
- P0: Admin viewer with filters.
- P1: Hash chaining of audit rows so tampering is detectable.

5.15 Statements and exports

- P1: Export transactions as CSV for a date range.
- P2: Monthly PDF statement.

6. Data model (summary)

- profiles: id, handle (unique, citext), full_name, us_address, role, status, created_at.
- accounts: id, owner_id, type (main, pot, system), currency, name, cached_balance (bigint), status, number, metadata.
- transactions: id, type, status, reference, parent_id, idempotency_key, created_by, created_at, posted_at, metadata (jsonb).
- ledger_entries: id, transaction_id, account_id, direction (debit or credit), amount (bigint, positive), currency, created_at. Append-only.
- idempotency_keys: key, user_id, request_hash, response (jsonb), created_at. Unique on user_id and key.
- merchants: id, name, category, icon.
- categories and category_rules: merchant to category mapping, with user overrides.
- pots: account_id, target_amount, target_date, emoji, rules (jsonb).
- pot_members: pot_id, user_id, role (for shared Pots).
- budgets: user_id, category, monthly_amount.
- limits: account_id, per_txn, daily, updated_by.
- risk_flags: id, transaction_id, rule, status, reviewed_by, reviewed_at.
- notifications: id, user_id, type, payload, read_at, created_at.
- audit_log: id, actor_id, action, target_type, target_id, ip, before, after, created_at, prev_hash, hash.
- reconciliation_runs: id, ran_at, discrepancies (jsonb), system_sum.

7. Architecture

- Frontend: Next.js (App Router) with TypeScript, Tailwind, deployed on Vercel. Server actions and route handlers for all mutations.
- Database and auth: Supabase Postgres with Supabase Auth and Realtime.
- Money logic lives in Postgres functions (for example post_transfer, post_purchase, move_to_pot), called via RPC from server-side code. Business-critical invariants are enforced by the database, not only by application code.
- Row level security on every table. Users can read only their own accounts, entries and notifications. All writes to ledger tables go through security definer functions. The service role key is used only on the server.

- Background work: Supabase scheduled functions or Vercel cron for scheduled transfers, auto-deposits, the nightly reconciliation job and simulated salary.
- Validation: Zod schemas on all inputs. Rate limiting on login and transfer endpoints.
- Observability: structured logs with a request id that matches the audit log entry.

8. Security requirements

- Integer money only. No floats in code, schema or JSON payloads.
- All balance-changing operations are atomic and idempotent.
- Deterministic lock ordering to avoid deadlocks.
- RLS on all tables, tested with a policy test suite that tries to read another user's data.
- Admin role checks server-side and in RLS.
- No secrets in the client bundle; environment variables per environment.
- Rate limiting, secure cookies, CSRF protection via framework defaults, strict input validation.
- Audit trail for every privileged action.

9. Testing strategy

This is where the project is won, so it gets its own section.

- Unit tests on money helpers: parsing, formatting, rounding rules, round-up calculation.
- Database tests (pgTAP or scripted SQL) for the ledger: unbalanced transaction rejected, update and delete on entries rejected, balance equals sum of entries.
- Concurrency test: parallel transfers from one account, parallel opposite transfers between two accounts (deadlock check), and repeated requests with the same idempotency key.
- Property-style test: generate random sequences of operations and assert the system-wide ledger sum is zero and no balance is negative.
- RLS tests: user A cannot read or write user B's data.
- End-to-end tests (Playwright) for sign up, fund, transfer, Pot and round-up flows.
- Reconciliation job run as part of the test suite and shown in the admin dashboard.

10. Non-functional requirements

- Transfers complete in under 500 ms at p95 for demo load.
- Fully responsive, designed mobile first, with US formatting (USD amounts, MM/DD/YYYY dates, US time zones).
- Accessible: keyboard navigation, contrast, labels.
- Clear empty, loading and error states on every screen.
- Consistent design system: one type scale, one spacing scale, a small component library.

11. Build plan (10 days)

Day 1: finalize this PRD, design tokens and key screens (home, send, pots), repo, CI, Supabase project, environments. Day 2: auth, onboarding, profiles, accounts, RLS foundation. Day 3: ledger schema, constraints, system accounts, sandbox top-up, post_transfer function. Day 4: transfers UI, idempotency, limits, error states, transaction feed and detail, realtime. Day 5: simulated purchases, categorization, Pots, round-ups. Day 6: notifications, velocity and large-transfer rules, held transfers. P0 complete by end of day. Day 7: admin console, audit log, reversals, reconciliation job. Day 8: tests: concurrency, property, RLS, end-to-end. Security pass. Fix what breaks. Day 9: P1 features (budgets, insights, payee list, CSV export), UI polish, seed data. Day 10: README with architecture decisions, demo video, deploy, final QA, buffer.

Cut order if time runs short: shared Pots, predictive alerts, PDF statements, push notifications, scheduled transfers, 2FA, in that order. Never cut tests on the ledger.

12. Demo script (3 minutes)

1. Sign up and onboarding, get an account (20 seconds).
2. Add money, simulate a few purchases, show the feed updating live with categories (40 seconds).
3. Send money to a second user in another window, show both feeds update instantly (30 seconds).
4. Double-click send to show idempotency: only one transfer posts (15 seconds).
5. Create a Pot, turn on round-ups, make a purchase, show the round-up land in the Pot (30 seconds).

6. Trigger a large transfer, show it held, approve it in the admin console, show the audit log (30 seconds).
7. Open the admin dashboard: system ledger sums to zero, reconciliation passes, concurrency test results (15 seconds).

13. Risks and mitigations

- Scope creep from cloning too much Monzo. Mitigation: the P0, P1 and P2 split and the cut order above.
- Subtle ledger bugs. Mitigation: database-level constraints, property tests, reconciliation job.
- Looking like a generic clone. Mitigation: a short README section on design decisions, plus one company-specific twist added after the core is stable.
- RLS misconfiguration leaking data. Mitigation: policy tests and a manual review pass on day 8.
- Time lost on UI polish early. Mitigation: build on a component library and polish on day 9 only.

14. Open questions

- Deadline and submission format: deployed link, repo, video, or all three?
- Does the company have a stack preference or a rubric?
- Should onboarding include a simulated SSN-style step to mirror real US KYC? Recommendation: skip it.
- Which company-specific twist should we add once the core is done?

15. README outline (deliverable)

1. What this is and what it deliberately is not.
2. Architecture diagram: client, server, Postgres functions, ledger.
3. The money model: double-entry, integer minor units, system accounts.
4. Concurrency and idempotency: how they are guaranteed.
5. Security model: RLS, roles, audit log.
6. Testing: what is covered and how to run it.
7. Scope decisions: what was cut and why.

8. Run locally, seed data, demo accounts.